

Défrichage

Chat Gpt entrées :

- Je suis un étudiant en première année de licence et je découvre le C. Je veux que tu me fasses un petit cours sur les structures en C. Jusque-là je n'ai vu que les variables, les conditions, les boucles, les fonctions, les pointeurs et les tableaux. Je ne veux pas l'allocation dynamique.

```
string nom1 = "Alice";  
string dateNaissance1 = "2001-04-12";  
int numSecu1 = 123456789;  
  
string nom2 = "Bob";  
string dateNaissance2 = "1999-11-30";  
int numSecu2 = 987654321;  
  
string nom3 = "Hicham";  
string dateNaissance3 = "2005-12-08";  
int numSecu3 = 215684813;  
  
trouverPlusVieux(dateNaissance1, dateNaissance2, dateNaissance3);
```

Quand les variables se multiplient...

Problèmes :

- Difficile de lire à qui appartient quelle date.
- Si on veut afficher le nom de la personne la plus âgée il faut recouper les infos "à la main"
 - Le code ne sait pas quelles infos vont ensemble !
- Si on ajoute une 4e personne, il faut modifier la fonction.

Nom
Prénom
Date de naissance
Numéro de rue
Nom de la rue
Code postal
Ville
Numéro de sécurité sociale

EtatCivil

```
EtatCivil individu1;  
EtatCivil famille[20];
```

Structurer, c'est donner du sens aux données

- Structures : encapsuler des variables
- Une structure correspond à un "concept".
 - Regroupe plusieurs variables de types différents qui décrivent une même entité
- Tas de variables éparpillées → **modèle de donnée** cohérent.
- Un nouveau type de variable est créé.

Le code devient lisible et réutilisable

```
string nom1 = "Alice";  
string dateNaissance1 = "2001-04-12";  
int numSecu1 = 123456789;  
  
string nom2 = "Bob";  
string dateNaissance2 = "1999-11-30";  
int numSecu2 = 987654321;  
  
string nom3 = "Hicham";  
string dateNaissance3 = "2005-12-08";  
int numSecu3 = 215684813;  
  
trouverPlusVieux(dateNaissance1, dateNaissance2, dateNaissance3);
```



```
EtatCivil personne1 = {"Alice", "2001-04-12", 123456789};  
EtatCivil personne2 = {"Bob", "1999-11-30", 987654321};  
EtatCivil personne3 = {"Hicham", "2005-12-08", 215684813};  
  
EtatCivil plusAgee = trouverPlusVieux(personne1, personne2, personne3);
```

- Le code “parle”. On comprend qu’on manipule des personnes.

- **trouverPlusVieux** manipule des *personnes*, pas des morceaux de données → moins d'erreurs !

→ **C’est ça, structurer** : regrouper, nommer, clarifier.

Définir une structure

Dans un fichier ".h" du nom de la structure.

Nom de la struct:

- Doit être un nom (pas un verbe)
- Pas de caractères spéciaux ni de _

struct se termine par ";"

Peut contenir des variables de tous types (dont struct, pointeurs, tableaux, etc.)

Tamagotchi.h

```
typedef struct
{
    int sante;
    int energie;
    double faim;
    double moral;

    Fight fightStats;
} Tamagotchi;
```

Données
membres /
attributs /
champs



Attention à ne pas oublier le ;

Accéder aux données membres de la structure

Tamagotchi.h

```
typedef struct
{
    int sante;
    int energie;
    double faim;
    double moral;

    Fight fightStats;
} Tamagotchi;
```

Main.c

```
#include "Tamagotchi.h" //Inclure le fichier ".h"

int main()
{
    Tamagotchi poupi;
    poupi.sante = 12; // NomInstance.NomDonneeMembre
    printf("La santé est %d\n", poupi.sante);

    poupi.fightStats.force = 50;

    //Tableau : rien à signaler de neuf
    // Type nomTableau[taille]
    Tamagotchi tamadex[151];
    tamadex[0].sante = 24;
}
```

NomAttribut

Initialisation à la déclaration

```
typedef struct
{
    int sante;
    int energie;
    double faim;
    double moral;
} Tamagotchi;
```

Initialisation séquentielle

Dans l'ordre de la définition des données

```
Tamagotchi poupi = {100,100,0.0,100.0};
```

Initialisation sélective

Le reste est initialisé à zéro ou à null (pointeurs)

```
Tamagotchi poupi = {.energie =100, .moral = 100.0};
```

Fonctions et structures

```
void afficherSante( Tamagotchi tama)
{
    printf("%d\n", tama.sante);
}

int main()
{
    Tamagotchi poupi = {100,100,0.0,100.0};
    afficherSante(poupi);
}
```

Utilisé comme n'importe quel autre type de variable

- La fonction reçoit une copie de la structure
- Les modifications n'impactent pas l'original

Associer des fonctions à des structures

Dans un fichier ".c" du nom de la structure

Inclure le fichier ".h"

Donne du contrôle

- Les structures regroupent les données,
- Les fonctions regroupent les comportements, permet de manipuler les structures.

→ Isoler la logique liée à une structure dans des fonctions dédiées.

Tamagotchi.c

```
#include "Tamagotchi.h"

void afficherSante( Tamagotchi tama)
{
    printf("%d\n", tama.sante);
}

Tamagotchi setSante ( Tamagotchi tama, int sante)
{
    if (sante < 0) {
        tama.sante = 0;
    }
    else if (sante > 100) {
        tama.sante = 100;
    }
    else {
        tama.sante = sante;
    }

    return tama;
}
```

Comme tout autre type on peut retourner une structure

Structures et pointeurs

- Structure : regroupe des informations cohérentes.
- Pointeur : manipule des informations sans tout recopier.
- Ensemble : gérer de grandes collections de données

Copie vs Pointeurs

```
Tamagotchi setSante ( Tamagotchi tama, int sante)
{
    if (sante < 0) {
        tama.sante = 0;
    }
    else if (sante > 100) {
        tama.sante = 100;
    }
    else {
        tama.sante = sante;
    }
    return tama;
}
```

```
int main()
{
    Tamagotchi poupi;
    poupi = setSante(poupi, 100);
}
```

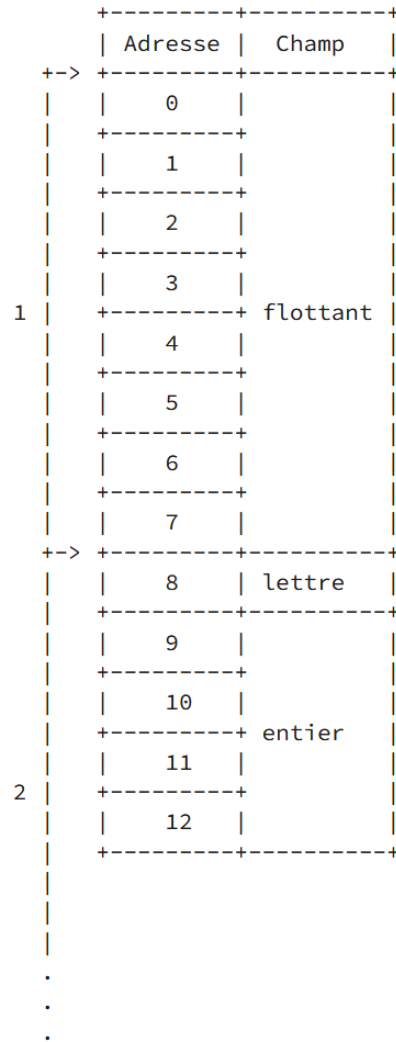


Deux copies

```
void setSante ( Tamagotchi * pTama, int sante)
{
    if (sante < 0) {
        (*pTama).sante = 0;
    }
    else if (sante > 100) {
        (*pTama).sante = 100;
    }
    else {
        (*pTama).sante = sante;
    }
}
```

```
int main()
{
    Tamagotchi poupi;
    setSante(&poupi, 100);
}
```

Manipuler un pointeur vers une structure



Une structure en mémoire

```
Tamagotchi* pTama;
Tamagotchi poupi;
pTama = &poupi;
```

```
*pTama.sante;           = *(pTama.sante)
    ↓
(*pTama).sante = 12;
```

Risque de
provoquer une
erreur

- `typedef struct { int x; int y; } Point;`
- Si on écrit `Point p = { 3 };`, alors `p.x == 3` et `p.y` n'est pas défini. ici y = 0
- Une fonction peut recevoir une structure en paramètre, mais cela crée une copie de toute la structure.
- Passer un pointeur vers une structure à une fonction permet de modifier la structure d'origine.
- On peut accéder à un champ de structure via un pointeur en utilisant `*ptr.champ`. il faut faire `(*ptr).champ`
- On peut retourner une structure depuis une fonction, mais c'est interdit si elle contient d'autres structures imbriquées. C'est tout à fait possible
- Un tableau statique de structures peut être déclaré comme :
`Point tab[10];`
- Si une structure contient un pointeur, alors l'initialisation partielle `{0}` met ce pointeur à NULL.
- On peut retourner une structure par valeur depuis une fonction, et cela copie l'intégralité de ses données.